Unification of APIs

Case	2.x Legacy Class	3.x Unified Class
Basic	Notifier<S>	**Notifier<S>**
	NotifierProvider<T, S>	
AutoDispose	AutoDisposeNotifier<S>	**Notifier<S>**
	NotifierProvider.autoDispose<T, S>	
Family	FamilyNotifier<S, Arg>	**Notifier<S>**
	NotifierProvider.family<T, S, Arg>	
AutoDispose + Family	AutoDisposeFamilyNotifier<S, Arg>	**Notifier<S>**
	NotifierProvider.autoDispose.family<T, S, Arg>	

Notifier / NotifierProvider (version 2)

```
class Counter extends Notifier<int> {
  @override
  int build() {
    return 0;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider<Counter, int>(Counter.new);
```

Notifier / NotifierProvider (version 3)

```
class Counter extends Notifier<int> {
  @override
  int build() {
    return 0;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider<Counter, int>(Counter.new);
```

AutoDisposeNotifier / NotifierProvider.autoDispose (version 2)

```
class Counter extends AutoDisposeNotifier<int> {
  @override
  int build() {
    return 0;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider.autoDipose<Counter, int>(
    Counter.new
  );
```

Notifier / NotifierProvider.autoDispose (version 3)

```
class Counter extends Notifier<int> {
  @override
  int build() {
    return 0;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider.autoDispose<Counter, int>(
    Counter.new
  );
```

FamilyNotifier / NotifierProvider.family (version 2)

```
class Counter extends FamilyNotifier<int, int> {
  @override
  int build(int arg) {
    return arg;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider.family<Counter, int, int>(
    Counter.new,
  );
```

Notifier / NotifierProvider.family (version 3)

```
class Counter extends Notifier<int> {
  Counter(this.arg);
  final int arg;

  @override
  int build() {
    return arg;
  }

  void increment() => state++;
}

final counterProvider =
  NotifierProvider.family<Counter, int, int>(
    Counter.new,
  );
```

AutoDisposeFamilyNotifier / NotifierProvider.autoDipose.family (version 2)

```
class Counter extends AutoDisposeFamilyNotifier<int, int> {
  @override
  int build(int arg) {
    return arg;
  }

  void increment() => state++;
}

final counterProvider = NotifierProvider
  .autoDispose.family<Counter, int, int>(
    Counter.new,
  );
```

Notifier / NotifierProvider.autoDispose.family (version 3)

```
class Counter extends Notifier<int> {
  Counter(this.arg);
  final int arg;

  @override
  int build() {
    return arg;
  }

  void increment() => state++;
}

final counterProvider = NotifierProvider
  .autoDispose.family<Counter, int, int>(
    Counter.new,
  );
```

Legacy Providers

- `StateProvider`, `StateNotifierProvider`, `ChangeNotifierProvider`
  - No longer recommended for use.
- In version 2
  - `import 'package:riverpod/riverpod.dart';`
- In version 3
  - `import 'package:riverpod/legacy.dart';`

Simplified Ref

- In version 2
  - `NotifierProviderRef`, `AsyncNotifierProviderRef`, `FutureProviderRef`, …

```
@riverpod
int example(ExampleRef ref) {...}
```

- In version 3
  - `Ref`

```
@riverpod
int example(Ref ref) {...}
```

Unification of state comparison operators

- In version 2, the criteria for determining state equality were not uniform across providers.
  - Some providers used '`identical`', while others used '`==`'.
  - In other words, some providers checked identical memory addresses, while others compared data content.
- In version 3, the standard was unified to '`==`'.
  - This small change had a significant impact on the performance of your app.

```
class User {
  User(this.name, this.age);
  final String name;
  final int age;
}

final oldUser = User(name: ‘Bob', age: 20);
final newUser = User(name: ‘Bob', age: 20);
print(identical(oldUser, newUser));  // false
print(oldUser == newUser); // false
```

```
class User {
  const User(this.name, this.age);
  final String name;
  final int age;

  @override
  bool operator ==(Object other) =>
    identical(this, other) ||
    other is User &&
    runtimeType == other.runtimeType &&
    other.name == name &&
    other.age == age;

  @override
  int get hashCode => name.hashCode ^ age.hashCode;
}
```

```
final oldUser = User(name: ‘Bob’, age: 20);
final newUser = User(name: ‘Bob’, age: 20);
print(identical(oldUser, newUser));  // false
print(oldUser == newUser); // true
```

```
class User extends Equatable {
  const User(this.name, this.age);
  final String name;
  final int age;

  @override
  List<Object> get props => [name, age];
}
```

```
@freezed
abstract class User with _$User {
  const factory User(
    String name,
    int age,
  ) = _User;
}
```

```
final oldUser = User(name: ‘Bob’, age: 20);
final newUser = User(name: ‘Bob’, age: 20);
print(identical(oldUser, newUser));  // false
print(oldUser == newUser); // true
```

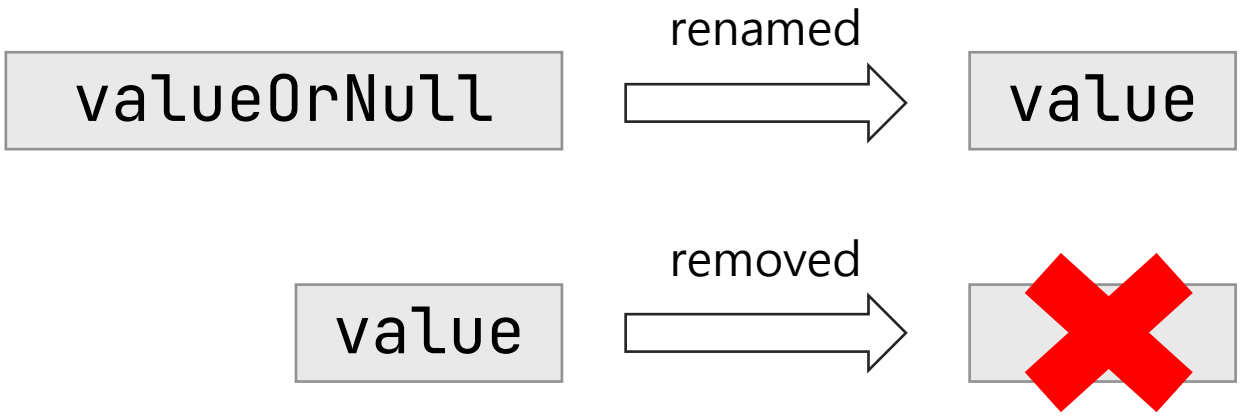
AsyncValue is now a sealed class

Riverpod 2

```
AsyncValue<int> value;
switch (value) {
  case AsyncData():
    print('data');
  case AsyncError():
    print('error');
  case AsyncLoading():
    print('loading');
 case _:
   print(‘default case’);
}
```

Riverpod 3

```
AsyncValue<int> value;
switch (value) {
  case AsyncData():
    print('data');
  case AsyncError():
    print('error');
  case AsyncLoading():
    print('loading');
  // No default case needed
}
```


valueOrNull
renamed
value
value
removed